

Université Mouloud Mammeri de Tizi-Ouzou
Faculté des Sciences — Département d'Informatique
Parcours Intelligence Artificielle

DOCUMENTATION TECHNIQUE

EducaRendez

Système de Gestion de Crèche
Application mobile de prise de rendez-vous

Réalisé par
Kenza Belkacem

Année universitaire
2025 – 2026

Table des matières

1	Présentation du Projet	2
1.1	Contexte	2
1.2	Fonctionnalités clés	2
2	Architecture Technique	3
2.1	Stack technologique	3
2.2	Architecture en couches	3
2.3	Patterns de conception utilisés	3
3	Base de Données	4
3.1	Schéma relationnel	4
3.2	Types de rendez-vous	4
3.3	Modèle Utilisateur — Pattern fromMap/toMap	4
4	Services Métier	5
5	Interface Utilisateur	5
5.1	Charte graphique	5
5.2	Différenciation par rôle	5
5.3	Flux de réservation	6
6	Compétences Démontrées	6
7	Perspectives d'Évolution	6
8	Conclusion	7

1 Présentation du Projet

1.1 Contexte

La gestion des crèches implique de nombreux rendez-vous entre parents, enseignants et administration. Les processus manuels génèrent des conflits horaires, des oublis et une charge administrative importante.

EducaRendez est une **application mobile native** développée avec **Flutter** qui digitalise la prise de rendez-vous au sein d'un établissement de petite enfance, en offrant une interface intuitive et un système de gestion multi-rôles.

1.2 Fonctionnalités clés

[T] 0.48 Gestion des comptes

- 3 rôles : Admin, Parent, Enseignant
- Authentification sécurisée par rôle
- Session utilisateur avec Singleton

Rendez-vous

- Création, consultation, annulation
- Vérification automatique des conflits
- 4 types de RDV pédagogiques

0.48 Disponibilités

- Créneaux configurables par l'admin
- Génération automatique des dates
- Activation/désactivation à la volée

Tableaux de bord

- Personnalisés selon le rôle
- Statistiques en temps réel
- Navigation fluide entre écrans

2 Architecture Technique

2.1 Stack technologique

Technologie	Version	Rôle
Flutter	3.x	Framework UI cross-platform (Android, iOS, Desktop)
Dart	≥ 3.0	Langage de développement
SQLite (sqflite_ffi)	2.3+	Base de données locale embarquée
intl	0.19+	Internationalisation et formatage de dates
Material Design 3	—	Système de design de Google

2.2 Architecture en couches

L'application suit une **architecture en 3 couches** séparant clairement la présentation, la logique métier et l'accès aux données :

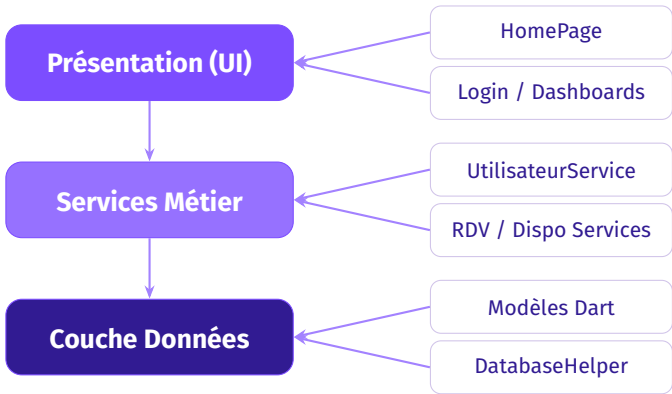


FIGURE 1 – Architecture en couches

2.3 Patterns de conception utilisés

Pattern	Application
Singleton	DatabaseHelper et Session : instance unique garantie pour la connexion DB et l'état utilisateur.
Repository	Services (UtilisateurService, RendezVousService) : abstraction entre la couche UI et l'accès aux données.
Factory	Utilisateur.fromMap(), RendezVous.fromMap() : construction des objets depuis les lignes SQLite.

3 Base de Données

3.1 Schéma relationnel

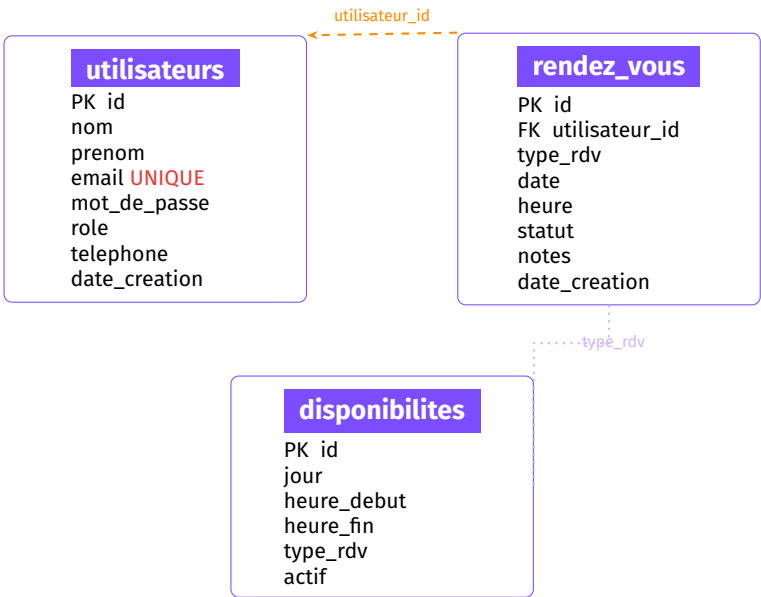


FIGURE 2 – Schéma relationnel (3 tables)

3.2 Types de rendez-vous

Clé	Libellé
rendez-vous_pedagogique	Rendez-vous pédagogique
entretien_pedagogique	Entretien pédagogique
embauche	Entretien d'embauche
formation	Formation

3.3 Modèle Utilisateur — Pattern fromMap/toMap

Sérialisation bidirectionnelle avec SQLite

```
class Utilisateur {
    int? id;
    String nom, prenom, email, motDePasse, role;
    String? telephone;
    DateTime dateCreation;

    // Constructeur avec parametres nommes requis
    Utilisateur({
        this.id, required this.nom, required this.prenom,
        required this.email, required this.motDePasse,
        required this.role, this.telephone, DateTime? dateCreation,
    }) : dateCreation = dateCreation ?? DateTime.now();

    // Serialisation vers SQLite
    Map<String, dynamic> toMap() => {
        'id': id, 'nom': nom, 'prenom': prenom,
        'email': email, 'motDePasse': motDePasse,
        'role': role, 'telephone': telephone,
        'dateCreation': dateCreation.toIso8601String(),
    };

    // Deserialisation depuis SQLite (Factory)
    factory Utilisateur.fromMap(Map<String, dynamic> map) =>
```

```

Utilisateur(
  id: map['id'], nom: map['nom'], prenom: map['prenom'],
  email: map['email'], motDePasse: map['motDePasse'],
  role: map['role'], telephone: map['telephone'],
  dateCreation: DateTime.parse(map['dateCreation']),
);
}

```

4 Services Métier

Service	Méthode	Description
4*Utilisateur	creer(u)	Insertion, retourne l'ID auto-incrémenté
	lireParEmail(e)	Recherche unique par email
	connexion(e,m)	Vérification email + mot de passe
	obtenirTous()	Liste complète (admin)
4*RendezVous	lireParUtilisateur(id)	RDV d'un parent, triés par date
	verifierDisponibilite(d,h,t)	Absence de conflit horaire
	creerRendezVous(rdv)	Insertion avec gestion d'erreur
	annulerRendezVous(id)	Passe le statut à annule
2*Disponibilite	obtenirActives()	Créneaux actifs triés par jour
	obtenirDates(j)	4 prochaines dates pour un jour donné

Algorithme de génération des dates

`obtenirDatesDisponibles` itère sur 28 jours, filtre par nom de jour en français, et s'arrête après 4 occurrences futures. Les dates passées sont exclues automatiquement.

5 Interface Utilisateur

5.1 Charte graphique

Élément	Valeur
Primaire	#7C4DFF
Primaire foncé	#311B92
Secondaire	#E040FB
Arrière-plan	#FAF5FF
Police	FiraSans (sans-serif)
Rayon bordure	12–25 pt selon le composant

5.2 Différenciation par rôle

L'écran d'accueil s'adapte dynamiquement selon le rôle connecté :

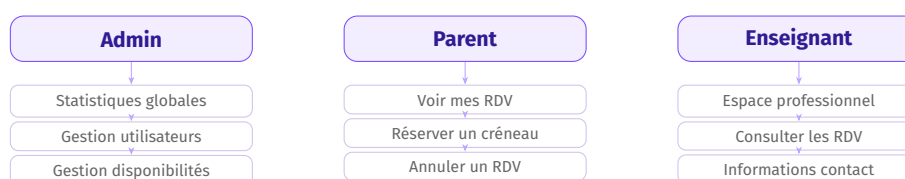


FIGURE 3 – Fonctionnalités par rôle

5.3 Flux de réservation

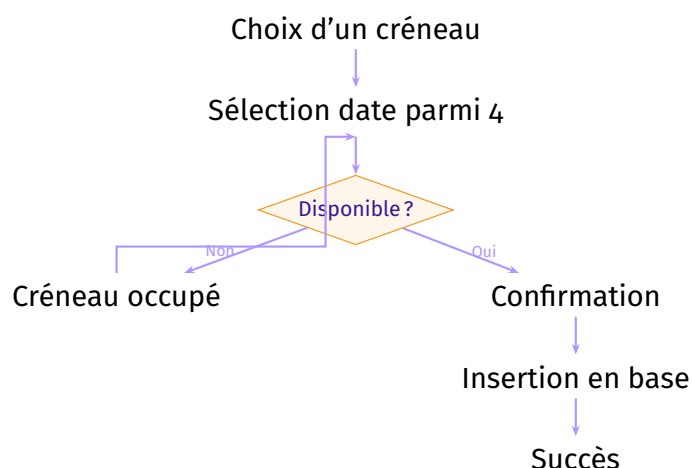


FIGURE 4 – Flux de réservation d'un rendez-vous

6 Compétences Démonstrées

Domaine	Compétences
Développement mobile	Flutter, Dart, widgets stateful/stateless, navigation, Material Design 3
Bases de données	SQLite, modélisation relationnelle, SQL, CRUD, clés étrangères, contraintes
Conception logicielle	Architecture en couches, patterns Singleton/Factory/Repository, séparation des responsabilités
Gestion de projet	Git, GitHub, documentation technique, structuration de projet

7 Perspectives d'Évolution

Feuille de route

- **v1.1** — Notifications push locales pour rappels de RDV
- **v1.2** — Persistance de session (`shared_preferences`)
- **v2.0** — Backend cloud (Firebase), authentification sécurisée (hachage `bcrypt`), multi-établissement
- **v2.1** — Module enfants : fiches, groupes, suivi de présence
- **v3.0** — Tableaux de bord analytiques, exports PDF, messagerie intégrée

Sécurité — Améliorations prioritaires

- Hachage des mots de passe (`bcrypt` au lieu du stockage en clair)
- Chiffrement de la base locale (`sqlcipher`)
- Validation côté serveur de toutes les opérations
- Gestion des permissions par rôle (middleware)

8 Conclusion

EducaRendez est un **prototype fonctionnel et abouti** qui démontre la maîtrise du développement mobile avec Flutter, de la conception de bases de données relationnelles et des principes d'architecture logicielle. L'application résout un problème réel — la gestion des rendez-vous en crèche — avec une approche rigoureuse : séparation en couches, patterns de conception éprouvés, interface soignée et code lisible.

Ce projet constitue une base solide pour aborder la licence L3 Informatique, en particulier le parcours Intelligence Artificielle, où les compétences en développement logiciel et en gestion de données structurées seront approfondies et mises en œuvre dans des contextes plus complexes.

Code source disponible sur GitHub

[https://github.com/kenzabelkacem-bit/
creche-management-system](https://github.com/kenzabelkacem-bit/creche-management-system)